Tribhuvan University
Institute of Engineering
Purwanchal Campus
पूर्वाञ्चल क्याम्पस

Software Requirements Specification

Project: Decentralized Smart Power Grid

Department of Computer & Electronics Engineering

Course	Software Engineering (Practical)
Course code	ENCT 352
Team	440 V
Submitted on	June 23, 2026

Team Members

S.N.	Name	Roll / ID
1	PRATIMA CHAUDHART	PUR080BCT064
2	RABIN KATTEL	PUR080BCT067
3	SABIN BASNET	PUR080BCT077
4	SARBESH TIMSINA	PUR080BCT082

Submitted to: Asst prof. Binay Lal Shrestha

Contents

1 Introduction

1.1 Purpose

This document specifies the software and hardware interfacing requirements for the Decentralized Prepaid Smart Power Grid system. It provides a comprehensive guide for the development team, project supervisor, and external evaluators to understand the functional and non-functional bounds of the system before structural implementation begins.

1.2 Scope

The system provides automated, transparent prepaid utility distribution using a simulated low-voltage IoT environment. It samples real-time consumer load profiles, computes usage fees, and enforces billing rules immutably via blockchain smart contracts. Additionally, integrated machine learning algorithms process data pipelines for predictive credit lifespans and anomaly-based power theft detection.

1.3 Definitions, Acronyms, and Abbreviations

SRS	Software Requirements Specification
AMI	Advanced Metering Infrastructure
ADC	Analog-to-Digital Converter
FR / NFR	Functional / Non-functional Requirement
RMSE	Root Mean Squared Error

1.4 Overview of the Document

This document defines the functional scope, behavioral models, operational constraints, and technical design metrics of the Decentralized Smart Power Grid application according to IEEE 830 standards.

2 Objectives

Aim: To elicit system requirements and compile an authoritative Software Requirements Specification using the IEEE 830 template.

Tools used: LaTeX for clean documentation and structural mathematical rendering.

3 Project Domain Analysis

3.1 Domain Description

Electrical utility infrastructure revolves around generation, high-voltage transmission, and localized distribution domains. The distribution layer relies heavily on tracking consumer energy consumption (measured in Watt-hours), validating ledger records, and executing billing routines. The domain encompasses two central structural entities: the Utility Administration and the Grid Consumer.

3.2 Existing System Overview

Conventional electrical utility setups rely on centralized Advanced Metering Infrastructure (AMI). Telemetry is piped into closed, private corporate databases managed entirely by a central authority. Consumers have no visibility into backend billing mathematics, and payment models operate predominantly on postpaid structures.

3.3 Problems in Existing System

The highly centralized architecture introduces deep systemic operational challenges:

- **Information Asymmetry:** Consumers cannot independently verify billing logs, causing trust deficits and hidden charge disputes.
- **Revenue Collection Risks:** Postpaid billing causes payment delays and frequent defaults, trapping utility companies in inefficient debt-recovery cycles.
- **Unresolved Distribution Leakages:** Line-tampering, meter-bypassing, and infrastructure anomalies go unflagged due to a lack of automated edge-analysis tools.
- **Data Integrity Risks:** Central databases are single-point-of-failure vulnerabilities open to internal manipulation or external security threats.

3.4 Proposed System

The proposed framework is a multi-layered decentralized ecosystem fusing IoT physical enforcement, localized middleware APIs, blockchain ledgers, and Machine Learning models. It features an automated prepaid smart contract system that isolates power lines instantly when balances are depleted, alongside a responsive user dashboard tracking live metrics and anomaly events.

4 Software Process Model

4.1 Waterfall Model

The classic linear Waterfall model is selected for this project: Requirements Analysis → System Design → Implementation → Testing → Deployment → Maintenance.

4.2 Process Model Diagram

Include necessary diagrams

4.3 Phases of the Model

1. **Requirements analysis:** Formulating this comprehensive IEEE 830 SRS document.
2. **System design:** Designing database models, pinning Solidity contract states, and detailing circuit schematics.
3. **Implementation:** Programming the ESP32 firmware ($C++$), configuring the FastAPI gateway (Python), and launching smart contracts.
4. **Testing:** Validating contract transactions, checking relay latency, and quantifying ML forecasting error.
5. **Deployment:** Spinning up the infrastructure locally via Hardhat and launching local host environments.
6. **Maintenance:** Optimizing smart contract execution costs (gas optimization) and refining ML feature variables.

4.4 Justification

- The transactional mechanics of prepaid utility metrics are stable and clearly defined.
- The dependencies between the physical hardware layer, contract layer, and middleware logic demand a predictable, sequential step-by-step pipeline.
- Fixed deadlines for academic checkpoints require highly deterministic progress markers.

5 Overall Description

5.1 Product Perspective

The system operates as a hybrid cyber-physical pipeline. The local ESP32 edge layer captures load telemetry and routes data to a FastAPI middleware backend, which queries and interacts securely with an immutable local blockchain ledger and a React analytical dashboard.

5.2 Product Functions

- **Automated Telemetry Processing:** Sampling continuous analog voltages to compute real-time power metrics.
- **Immutable Transaction Management:** Handling token balance updates directly on-chain using secure deduction math.
- **Autonomous Edge Control:** Instantly opening hardware relays to isolate load lines when credit profiles drop to zero.
- **Predictive Budget Forecasting:** Applying linear regression to project exact user balance depletion horizons.
- **Grid Anomaly Detection:** Using Isolation Forest algorithms to flag sudden line-tampering anomalies.

5.3 User Characteristics

- **Grid Consumer:** Views real-time demand figures, reads live remaining credit time-frames, and tops up token balances.
- **Grid Administrator:** Monitors macro-level distribution feeds, tracks network transaction latency, and acts on real-time theft warning flags.

5.4 Key Entities

- **Smart Meter Node:** Unique MAC/Device ID, connected hardware address, real-time load, status state (Active/Inactive).
- **Consumer Wallet:** Cryptographic wallet address, remaining credit balance (Bal_{token}), allocated device link.
- **Telemetry Frame:** $Load_{curr}$, $Energy_{cum}$, Δ_{load}, $Hour_{tod}$.
- **System Ledger Record:** Unique blockchain transaction hash, deduction amount, timestamp.

5.5 Constraints

- The hardware node must rely entirely on the operational constraints of the ESP32 processing chip.
- All blockchain calculations must operate within a local test environment (Hardhat) to bypass live transaction network costs.

5.6 Assumptions and Dependencies

- The backend engine relies on a persistent connection to the blockchain environment via Web3.py.
- The physical edge device requires a reliable Wi-Fi network connection to stream JSON packets to the API gateway.

6 Specific Requirements

6.1 External Interface Requirements

6.1.1 User Interfaces

A responsive web application consisting of a Consumer View (featuring live energy gauges, transaction histories, and credit countdown elements) and an Admin View (displaying real-time anomaly warning flags).

6.1.2 Hardware Interfaces

- ESP32 Microcontroller.
- Linear Potentiometer (for analog grid load simulation).
- Electromechanical Relay (for circuit line disconnection).
- I2C LCD Display (for localized device status metrics).

6.1.3 Software Interfaces

- **Database:** SQLite for quick historical logging.
- **Blockchain Dev-Env:** Hardhat for running localized Solidity testnets.
- **ML Environment:** Scikit-Learn (Python) for execution of regression and Isolation Forest pipelines.

6.1.4 Communication Interfaces

- HTTP REST protocol for ESP32 data ingestion into FastAPI.
- WebSockets for streaming live telemetry updates to the React interface.
- JSON-RPC protocol for backend communication with the blockchain ledger via Web3.py.

6.2 Functional Requirements

ID	Functional Requirement	Priority
FR-1	The ESP32 node shall continuously sample load data and transmit it to the backend as structured JSON data packets.	High
FR-2	The smart contract shall execute immutable token balance deductions based on consumption telemetry.	High
FR-3	The system shall immediately isolate the physical relay if the user's smart contract credit balance drops to ≤ 0.	High
FR-4	The system shall compute a time-to-exhaustion projection using a supervised Linear Regression framework.	Medium
FR-5	The machine learning engine shall flag potential grid theft if an abrupt, unprompted drop (Δ_{load}) occurs without a break.	Medium
FR-6	The system shall update the localized on-chip LCD screen to reflect live power usage and balance status changes.	Low

6.3 Non-functional Requirements

ID	Non-functional Requirement	Category
NFR-1	The smart contract transaction confirmation and billing logic run-time must update within 3 seconds of telemetry ingestion.	Performance
NFR-2	Grid circuit isolation commands must trigger the physical relay within 500 milliseconds of contract exhaustion confirmation.	Performance
NFR-3	Consumer smart contract states and account keys must be managed through secure, cryptographically validated signatures.	Security
NFR-4	The predictive budgeting algorithm must maintain an evaluation accuracy boundary below an acceptable Root Mean Squared Error (RMSE) limit.	Reliability

6.4 Other Requirements

None.

7 Conclusion

In this lab, the development team analyzed, mapped, and structured the specifications for the Decentralized Prepaid Smart Power Grid system based on the IEEE 830 template. The resulting functional, non-functional, and physical configuration constraints provide a precise architectural roadmap that will be directly traced throughout the subsequent system design, development, and testing phases.

8 References

1. IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
2. Smith, J., & Kumar, A. (2022). IoT-Based Smart Energy Meter for Real-Time Monitoring and Automated Billing. International Journal of Smart Grid Technologies, 10(3), 150-158.
3. J. Jithish, B. Alangot, N. Mahalingam, and K. S. Yeo, "Distributed Anomaly Detection in Smart Grids: A Federated Learning-Based Approach," IEEE Access, vol. 11, pp. 9824-9838, 2023.